Name: _____ Register No: _____

Experiment 18

Question: Experiment to understand KNN algorithm for a given data set.

Aim

To understand the K-Nearest Neighbours (KNN) algorithm and classify the Iris data set.

Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

def exp_18():
    from sklearn.datasets import load_iris
    from sklearn.model_selection import train_test_split
    from sklearn.preprocessing import StandardScaler
    from sklearn.neighbors import KNeighborsClassifier
    from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

    iris = load_iris()
    X, y = iris.data, iris.target
    names = iris.target_names
    print("Dataset: Iris | samples =", X.shape[0], "| features =", X.shape[1])
    print("Classes :", [str(n) for n in names], "\n")

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.3, random_state=42, stratify=y)
    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train)
    X_test = scaler.transform(X_test)

    # Find the best K
    k_values = range(1, 16)
    acc = []
    for k in k_values:
        knn = KNeighborsClassifier(n_neighbors=k)
        knn.fit(X_train, y_train)
        acc.append(accuracy_score(y_test, knn.predict(X_test)))
    best_k = k_values[int(np.argmax(acc))]
    print("Accuracy for K = 1 to 15:")
    print({k: round(a, 3) for k, a in zip(k_values, acc)})
    print("Best K =", best_k, "with accuracy", round(max(acc), 4), "\n")

    knn = KNeighborsClassifier(n_neighbors=best_k)
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    print("Test accuracy:", round(accuracy_score(y_test, y_pred), 4))
    cm = confusion_matrix(y_test, y_pred)
    print("\nConfusion matrix:\n", cm)
    print("\nClassification report:\n", classification_report(y_test, y_pred, target_names=names))

    sample = scaler.transform([[5.1, 3.5, 1.4, 0.2]])
    print("Prediction for [5.1, 3.5, 1.4, 0.2] ->", names[knn.predict(sample)[0]])

    fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
    ax[0].plot(list(k_values), acc, marker="o")
    ax[0].axvline(best_k, color="red", linestyle="--", label=f"Best K = {best_k}")
    ax[0].set(title="Accuracy for different K", xlabel="K (number of neighbours)",
        ylabel="Accuracy")
    ax[0].legend()
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=names, yticklabels=names,
        ax=ax[1])
    ax[1].set(title="Confusion Matrix", xlabel="Predicted", ylabel="Actual")
    plt.tight_layout()
    plt.show()


exp_18()
```

Output

```
Dataset: Iris | samples = 150 | features = 4
```

```
Classes : ['setosa', 'versicolor', 'virginica']

Accuracy for K = 1 to 15:
{1: 0.933, 2: 0.889, 3: 0.911, 4: 0.911, 5: 0.911, 6: 0.911, 7: 0.933, 8: 0.911, 9: 0.956, 10:
    0.933, 11: 0.956, 12: 0.956, 13: 0.933, 14: 0.956, 15: 0.933}
Best K = 9 with accuracy 0.9556

Test accuracy: 0.9556

Confusion matrix:
 [[15  0  0]
 [ 0 15  0]
 [ 0  2 13]]

Classification report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        15
  versicolor       0.88      1.00      0.94        15
   virginica       1.00      0.87      0.93        15

    accuracy                           0.96        45
   macro avg       0.96      0.96      0.96        45
weighted avg       0.96      0.96      0.96        45

Prediction for [5.1, 3.5, 1.4, 0.2] -> setosa
```

Graph

